

■ Day 2 — Setup & Skeleton

Recap card for Task Manager with Login (ASP.NET Core MVC). Companion to the 12-Day Build Recipe — read this after finishing Day 2, before opening Day 3.

Goal for the day

End of day: a running ASP.NET Core app with Identity scaffolded, your **TaskItem** entity sketched on paper, and one real git commit that isn't just the bare template.

Block 1 (0:00–0:15) — Reconnect, no coding

- Reopen Day 1 diagrams — architecture layers (1.1) and the ER diagram (1.4).
- Explain out loud or in **LEARNING.md**: what is a DbContext? What does `-au Individual` actually add to the project? If you can't answer, that's fine — it's what today teaches.
- Open a fresh LEARNING.md entry stub for today.

Block 2 (0:15–1:00) — Scaffold and explore

```
dotnet new mvc -o TaskManager -au Individual
cd TaskManager
dotnet run
```

- Confirm it loads in the browser. Click Register, Login — just to see the pages exist.
- **Stop and explore before writing anything.** Open these files and spend 5 min each just reading:
 - `Program.cs` — find where `AddDefaultIdentity` and `AddDbContext` get called
 - `Areas/Identity/Pages/Account/` — see what Identity scaffolded for you
 - `appsettings.json` — find the connection string

Write one sentence per file in LEARNING.md: what it's for.

■■ This is the "20-minute rule" — struggle with reading the structure before asking anything.

Block 3 (1:00–1:45) — Entity design (paper first)

- On paper, sketch TaskItem: Id, Title, Description, DueDate, IsCompleted, CreatedAt, CompletedAt, Priority, UserId.
- Decide: SQLite (simplest, recommended) or SQL Server. Write down *why* in LEARNING.md.
- Install NuGet packages: `Microsoft.EntityFrameworkCore.Sqlite`, `.Tools`, `.Design`. Don't write the model class yet — that's Day 3.

Block 4 (1:45–2:15) — Poke at what Identity built

- Register a real test user through the UI.
- Open the SQLite `.db` file in a DB browser (run `dotnet ef database update` first if no DB exists yet — just note that and move on).

- Find the `AspNetUsers` table. Look at the columns. This is where diagram 1.4 stops being abstract.

Block 5 (2:15–2:45) — AI as reviewer, not typist

Now, and only now, bring questions like:

- "I registered a user — walk me through what happened between clicking Register and the row appearing in `AspNetUsers`."
- "What's the difference between `dotnet ef migrations add` and `dotnet ef database update` — I haven't run either yet, why not?"

■ Don't ask for `TaskItem.cs` yet. Ask about what you already did, not for new code.

Block 6 (2:45–3:00) — Blank-Page Test + commit

- Close the code. From memory, list: the 5 files Identity added, the 9 columns `TaskItem` will have, why you picked your DB.
- Finish `LEARNING.md`: What I built · What confused me · What I now understand · Question for later.

```
git add .
git commit -m "scaffold app with Identity, sketch TaskItem entity"
```

Definition of Done — Day 2

- App runs, Register/Login pages load
- Registered a test user and found the row in `AspNetUsers`
- EF Core packages installed
- `TaskItem` sketched on paper with all 9 fields
- `LEARNING.md` has a real Day 2 entry
- One meaningful git commit

Don't do yet

Don't write `TaskItem.cs` or touch `ApplicationDbContext.cs` today — that's Day 3, and it depends on today's DB choice being settled. Don't ask for the model code either; typing it yourself on Day 3 forces you to actually decide types and attributes instead of pattern-matching.

■ Core discipline reminder: diagrams are your map, the Explain-First Loop is your method, struggle-before-AI is your gate. Type everything yourself; use AI to explain, review, and challenge — never to produce.

Next: Day 3 — Data modeling with EF Core